Marion Lardeux
Aurore Le Manac’h

ESIR2 IoT
08/11/2026

ESIR 2 IOT - OMD S7

COMPTE-RENDU TP2

Mini-éditeur

Université de Rennes

ESIR ECOLE SUPERIEURE D'INGENIEURS DE RENNES

Introduction :

Dans le cadre de ce TP, nous avons réalisé à la fois la conception et le développement d'un mini-éditeur de texte, avec deux versions successives pour une programmation dite "en spirale". Notre système devait mettre en place différentes fonctions usuelles pour l'utilisateur comme la saisie et la suppression de texte, les actions de coupe, copie et collage ainsi que l'annulation, le rétablissement et l'enregistrement et rejeu d'actions. Les objectifs de ce TP étaient à la fois de réaliser l'analyse d'un système pour lequel nous avions la majorité des détails à décider, mais aussi d'allier la programmation à la conception pour créer un système réalisable et en accord avec les diagrammes conçus.

I - Dossier de conception

I.1 - Cas d'utilisation

I.1.a) Diagramme

Nous avons identifié comme unique acteur de notre système l'utilisateur/user, qui va se servir de l'éditeur. Dans la première version du système, il peut déplacer le curseur ou les bornes de sélection, copier/couper/coller la sélection et saisir/supprimer du texte. Dans la seconde version s'ajoutent l'annulation et le rétablissement ainsi que l'enregistrement et le rejeu des actions effectuées. Nous obtenons alors ce diagramme de cas d'utilisation :

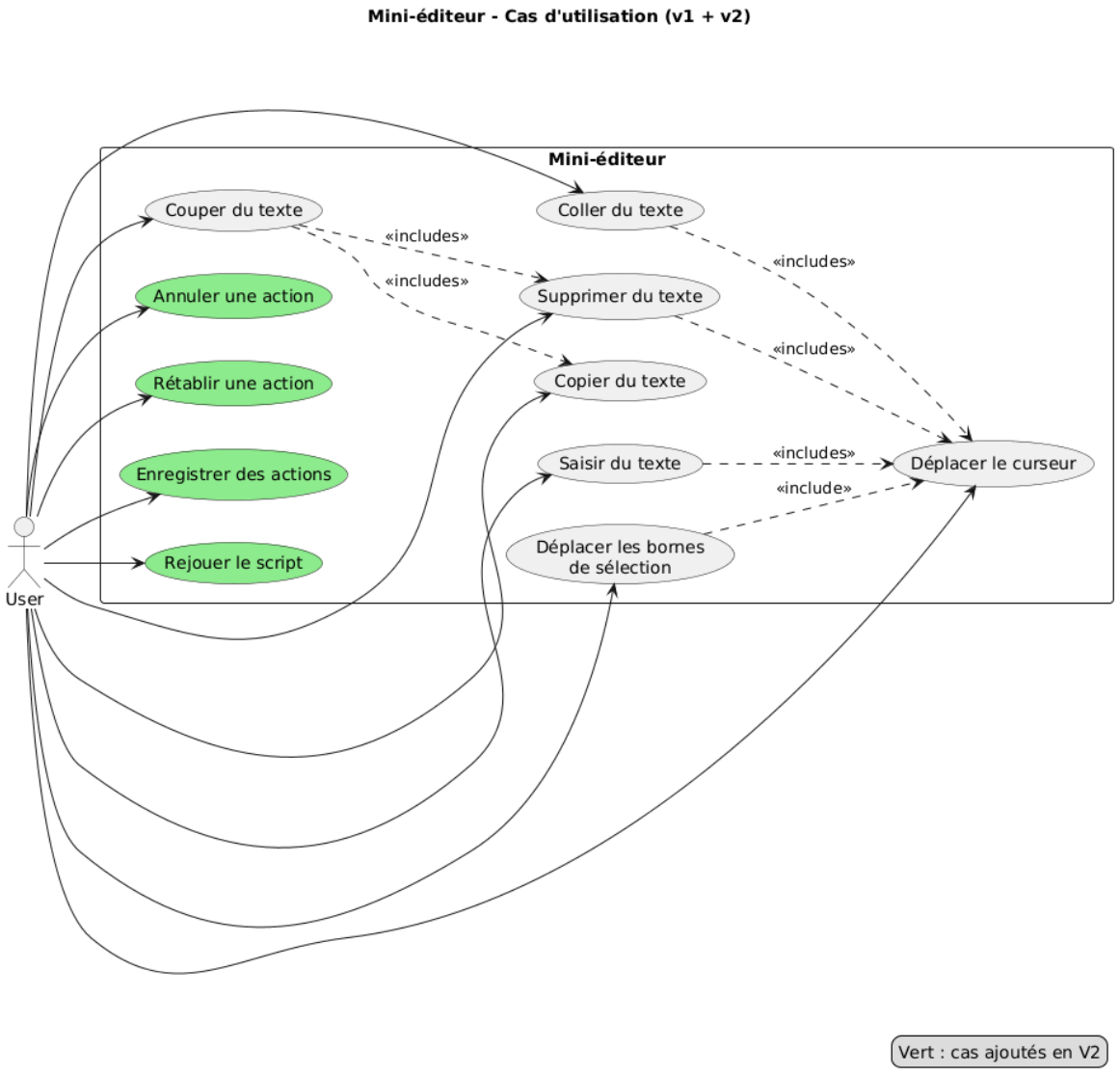


Figure 1 : Diagramme de cas d'utilisation (les actions ajoutées dans la v2 figurent en vert)

Nous avons pris différentes décisions concernant la mise en pratique de ces cas d'utilisation.

Décisions dans la v1 (déplacer, saisir, supprimer, copier, couper, coller) :

- Déplacer le curseur hors du texte : le curseur reste à la position extrême du texte et ne peut pas la dépasser
- Supprimer avec un éditeur vide : l'éditeur ne change pas, le curseur reste à sa position initiale
- Copier/Couper avec une sélection vide : l'éditeur et le presse-papier ne changent pas
- Coller avec une sélection vide : le contenu du presse-papier est inséré sur la position du curseur qui est ensuite déplacé à la fin du texte collé
- Coller avec une sélection non vide : le contenu de la sélection est supprimé, puis le contenu du presse-papier est inséré et le curseur est placé à la fin du texte ajouté
- Coller avec un presse papier-vide : l'éditeur et le presse-papier ne changent pas

Décisions dans la v2 (défaire, rétablir, enregistrer, rejouer) :

- Défaire : le presse-papier ne change pas, il n'est pas restauré à son état précédent
- Action après un défaire : la pile des actions à rétablir est vidée, elles sont impossibles à restaurer
- Rejeu : les actions rejouées ne sont pas enregistrées de nouveau

I.1.b) Descriptions, scénario

Nous avons élaboré des scénarios et descriptions textuelles de chacun des cas d'utilisation évoqués.

Déplacer le curseur :

- But : Déplacer le curseur sur le texte de l'éditeur
- Début : L'utilisateur souhaite modifier la position du curseur sur le texte
- Fin : Le curseur est à la position voulue
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur appuie sur les touches ← ou →
    2) Le curseur se déplace sur le texte, un caractère à la fois
- Scénario d'exception :
    1) Si le curseur arrive à une extrémité du texte, il ne peut pas aller plus loin dans cette direction

Déplacer la borne de sélection :

- But : Sélectionner du texte pour effectuer des actions dessus
- Début : L'utilisateur souhaite sélectionner du texte ou modifier la sélection
- Fin : Le texte souhaité est sélectionné
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur déplace le curseur à la position souhaitée (correspondant à une extrémité de la sélection)
    2) Il appuie sur 'Maj' et sur les touches ← ou →
    3) La borne de sélection part du curseur et se déplace, un caractère à la fois
    4) Une sélection se crée lorsque la borne de sélection est à un emplacement différent du curseur
- Scénario alternatif :
    1) Si, après avoir déplacé la borne dans une direction, l'utilisateur appuie sur la flèche opposée, alors cela réduit la sélection d'un caractère (la borne de sélection fait demi-tour)
    2) Si, après avoir déplacé la borne dans une direction, l'utilisateur fait demi-tour jusqu'à atteindre la position du curseur et la dépasse dans la direction opposée, cela crée une sélection partant du curseur et s'étendant dans l'autre sens

- Scénario d'exception :
    1) Si la borne atteint une extrémité de la zone du texte, alors la sélection ne peut plus s'étendre de ce côté

Saisir du texte :

- But : Ajouter du texte à l'éditeur
- Début : L'utilisateur souhaite écrire du texte
- Fin : Le texte saisi est ajouté au texte existant
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur déplace le curseur à l'endroit où il veut ajouter le texte
    2) Il entre le texte à saisir
    3) Le système ajoute le texte au buffer
    4) Le curseur est placé à la fin du texte ajouté
- Scénario alternatif :
    1) Si, avant la saisie, du texte était sélectionné, alors le texte ajouté le remplace dans le buffer
- Scénario d'exception :
    1) Si les touches saisies ne sont pas des caractères et ne correspondent à aucune action, rien ne se passe

Supprimer du texte :

- But : Retirer du texte de l'éditeur
- Début : L'utilisateur souhaite supprimer du texte
- Fin : Le texte est retiré de l'éditeur
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur déplace le curseur à l'endroit où il veut retirer du texte
    2) Il appuie sur 'Retour arrière'
    3) Le texte à gauche du curseur est supprimé, un caractère à la fois
    4) Le curseur est décalé vers la gauche à chaque suppression, un caractère à la fois
- Scénario alternatif :
    1) Si du texte est sélectionné, alors toute la sélection est supprimée pour la première action de suppression et la sélection redevient un curseur situé à l'emplacement de la sélection
- Scénario d'exception :
    1) S'il n'y a pas de texte dans le buffer, alors rien ne se passe

Copier du texte :

- But : Copier du texte dans le presse-papier
- Début : L'utilisateur souhaite ajouter du texte au presse-papier
- Fin : Le texte sélectionné est dans le presse-papier
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur réalise la sélection du texte qu'il souhaite copier
    2) Il réalise la commande de copie (Ctrl+C)
    3) Le système ajoute la sélection au presse-papier
- Scénario d'exception :
    1) Si le texte sélectionné est vide, alors il n'y a aucun changement dans le presse-papier

Couper du texte :

- But : Copier du texte dans le presse-papier et le retirer de l'éditeur
- Début : L'utilisateur souhaite ajouter du texte au presse-papier et le supprimer

- Fin : Le texte sélectionné est dans le presse-papier et retiré du buffer
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur réalise la sélection du texte qu'il souhaite ajouter au presse-papier
    2) Il réalise la commande de coupe (Ctrl+X)
    3) Le système ajoute la sélection au presse-papier
    4) Il la supprime du buffer
    5) La sélection redevient un curseur situé à l'emplacement de la sélection
- Scénario alternatif :
    1) Si le texte sélectionné est vide, alors il n'y a aucun changement dans le presse-papier ou dans le buffer

Coller du texte :
- But : Ajouter à l'éditeur le texte contenu dans le presse-papier
- Début : L'utilisateur souhaite ajouter le contenu du presse-papier à l'éditeur
- Fin : Le texte est ajouté au buffer
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur déplace le curseur à l'endroit où il veut ajouter le texte
    2) Il réalise la commande de collage (Ctrl+V)
    3) Le système ajoute le texte du presse-papier au buffer
    4) La sélection redevient un curseur situé à la fin du texte collé
- Scénario alternatif :
    1) Si la sélection est non vide, alors celle-ci est remplacée par le contenu du presse papier lors du collage

Parmi les actions implémentées dans la v1 du mini-éditeur, certaines ont une étape de plus dans la v2, il s'agit de l'ajout au sommet de la pile des actions à défaire : la saisie, la suppression, la coupe et le collage. Le déplacement du curseur est ignoré par l'action d'annulation. La copie n'affecte que le presse-papier, or son état n'est pas restauré lors d'une annulation, donc cette action n'est pas ajoutée non plus (seule la potentielle sélection de texte précédant à la copie est prise en compte).

Défaire une action :
- But : Revenir en arrière et annuler la dernière action effectuée
- Début : L'utilisateur souhaite annuler sa dernière action dans l'éditeur
- Fin : Le buffer est de retour à son état précédent
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur réalise la commande défaire (Ctrl+Z)
    2) Il retourne en arrière sur la partie impactant le buffer de la dernière action réalisée (le presse-papier ne change pas) :
        - S'il s'agissait d'une saisie, le système supprime le dernier caractère ajouté
        - S'il s'agissait d'une suppression, le système remet à son emplacement dans le buffer le dernier caractère supprimé
        - S'il s'agissait d'une sélection (pour une copie ou toute autre action), le curseur retourne soit à la position de l'action précédente (saisie, suppression, collage) ou bien redevient la sélection précédente selon l'état précédent (curseur ou sélection)
        - S'il s'agissait d'une coupe, le système remet le texte à son emplacement dans le buffer
        - S'il s'agissait d'un collage, le système supprime le texte du buffer
    3) L'action est retirée de la pile des actions à annuler
    4) Elle est ajoutée au sommet de la pile des actions à rétablir

- Scénario alternatif :
    1) S'il s'agissait d'un collage et que du texte avait été remplacé, celui-ci est rétabli dans le buffer et les bornes de sélection sont replacées de part et d'autre de ce texte
- Scénario d'exception :
    1) Si aucune action n'est à annuler, rien ne se passe

Rétablir une action :

- But : Revenir en avant et refaire la dernière action annulée
- Début : L'utilisateur refaire sa dernière action annulée
- Fin : Le buffer est de retour à son état avant la dernière annulation
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur réalise la commande rétablir (Ctrl+Y)
    2) Il retourne en avant sur la partie impactant le buffer de la dernière action annulée (le presse-papier ne change pas) :
        - S'il s'agissait d'une saisie, le caractère est de nouveau ajouté à son emplacement dans le buffer
        - S'il s'agissait d'une suppression, le caractère est de nouveau retiré du buffer
        - S'il s'agissait d'une sélection (pour une copie ou toute autre action), le curseur retourne soit à la position de l'action suivante (saisie, suppression, collage) ou bien redevient la sélection suivante selon l'état précédent (curseur ou sélection)
        - S'il s'agissait d'une coupe, le système supprime de nouveau le texte du buffer
        - S'il s'agissait d'un collage, le système remet le texte à son emplacement dans le buffer
    3) L'action est ajoutée au sommet de la pile des actions à annuler
- Scénario alternatif :
    1) S'il s'agissait d'un collage avec une sélection non vide, alors à ce stade la sélection est sur le texte qui avait été remplacé donc rétablir l'action implique de remplacer de nouveau ce texte par le contenu du presse papier au moment de l'action initiale
- Scénario d'exception :
    1) Si aucune action n'a été annulée depuis le dernier rétablissement, rien ne se passe

Enregistrer des actions :

- But : Garder en mémoire une suite d'actions sur l'éditeur
- Début : L'utilisateur souhaite enregistrer des actions
- Fin : Les actions sont en mémoire
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur réalise la commande enregistrer (Ctrl+R) pour démarrer l'enregistrement
    2) Il réalise les actions de son choix sur le texte
    3) Il réalise une seconde fois la commande enregistrer pour terminer l'enregistrement
    4) Le système garde en mémoire les actions réalisées entre les deux activations de la commande

Rejouer des actions :

- But : Faire réaliser la suite d'actions enregistrée par le système
- Début : L'utilisateur souhaite reproduire une série d'actions avec une simple commande
- Fin : La série d'actions a été reproduite à l'emplacement du curseur
- Acteurs : User
- Scénario nominal :
    1) L'utilisateur enregistre une série d'actions avec la commande d'enregistrement
    2) Il positionne le curseur à l'endroit du texte où il souhaite reproduire les actions

3) Il réalise la commande de rejeu (Ctrl+P)
4) Le système retrace les actions enregistrées et les applique

- Scénario alternatif :
  1) Si la position du curseur ne permet pas de reproduire exactement les actions, celles-ci sont réalisées en respectant les contraintes énoncées précédemment donc le résultat peut varier d'une position à l'autre (par exemple si le curseur est au début d'un texte et qu'il doit être décalé vers la gauche, l'action sera ignorée)
- Scénario d'exception :
  1) Si aucune action visible (hors déplacements) n'est effectuée durant l'enregistrement, rien ne se passe lors du rejeu

I.2 - Classes

I.2.a) Diagramme

Nous avons réalisé le diagramme de classes pour chacune des deux versions :

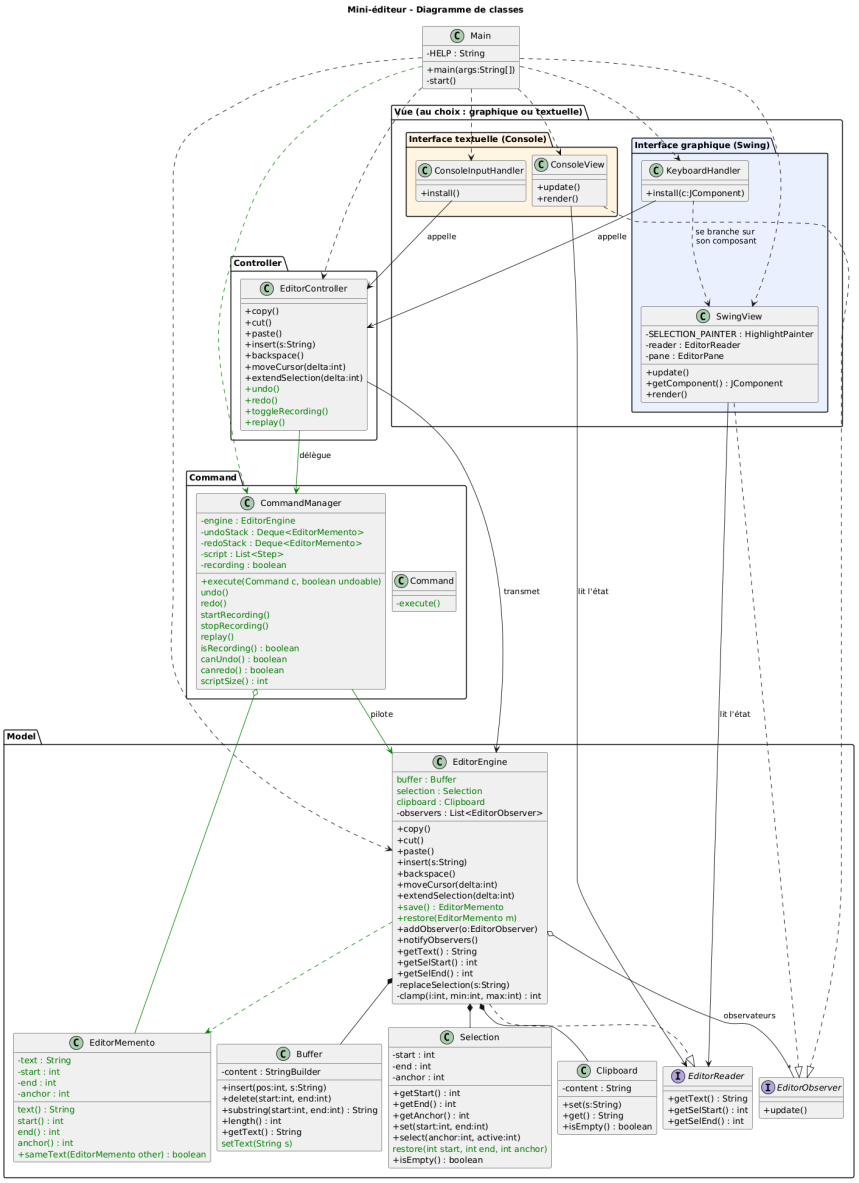


Figure 2 - Diagramme de classes (les méthodes et classes ajoutées dans la v2 figurent en vert)

Le package Vue gère tout ce qui concerne l'interface graphique et la saisie au clavier. En l'occurrence, la classe SwingView réalise l'opération de rendering pour l'éditeur graphique. La classe KeyboardHandler appelle les différentes fonctions d'EditorController (du package Contrôleur) selon le

raccourci clavier détecté. Ces appels sont transmis au package Modèle via la classe EditorEngine. Cette classe contient une instance de Buffer (le contenu de l'éditeur), une instance de Selection (l'état des bornes de sélection) et une instance de Clipboard (le contenu du presse-papier). C'est aussi EditorEngine qui manipule la position du curseur. Nous avons également des interfaces responsables de récupérer les données de l'éditeur et de traquer les modifications pour mettre à jour le rendering.

Dans la v2, nous avons à la fois ajouté les différentes actions supplémentaires dans EditorController et EditorEngine, et nous avons ajouté des méthodes pour rétablir l'état de la sélection ou du buffer lors d'une action défaire/rétablir. De plus, une nouvelle classe, EditorMemento, a été ajoutée. Il s'agit de la classe responsable de sauvegarder l'état de l'éditeur et de la sélection au fur et à mesure des étapes, pour les ajouter aux piles des actions à défaire et à rétablir.

I.2.b) Descriptions

Nous avons construit le diagramme de classe en effectuant des choix de conception et de modélisation expliqués ci-dessous pour chacune des classes. Pour ce qui est de l'interface, nous avons ajouté au diagramme les deux conceptions : l'interface graphique avec swing et l'interface textuelle dans la console. En pratique, nous avons programmé une interface graphique.

Interface graphique :
La classe **SwingView** implémente la méthode update() de l'interface EditorObserver. Lorsqu'elle est appelée, cette méthode lit le texte du buffer et la portion sélectionnée et les affiche dans une fenêtre Swing. Elle se branche à la saisie clavier en exposant son composant via la méthode getComponent().
Cette classe n'évolue pas avec la v2.

La classe **KeyboardHandler** convertit les raccourcis clavier en actions sur l'éditeur : caractère = insert(), retour arrière = backspace(), flèches = moveCursor(), maj + flèches = extendSelection(), ctrl + x = cut(), ctrl + c = copy(), ctrl + v = paste().
Dans la v2 sont reconnus en plus les raccourcis suivants : ctrl + z = undo(), ctrl + y = redo, ctrl + r = toggleRecording(), ctrl + p = replay.

Interface textuelle :
La classe **ConsoleView** est l'équivalent de SwingView pour l'interface graphique. Elle implémente les méthodes update() et render() pour mettre à jour le contenu de l'éditeur et l'afficher sur la console.
Cette classe n'évolue pas avec la v2.

La classe **ConsoleInputHandler** est l'équivalent de KeyboardHandler pour l'interface graphique. Elle permet de convertir les raccourcis clavier en appels à des méthodes qu'elle transfère à EditorController.

La classe **Buffer** contient le texte affiché sur l'éditeur. Il est possible de récupérer le contenu, d'insérer, de supprimer, d'extraire une portion et de calculer la longueur de ce texte.
Dans la v2, une nouvelle méthode setText() permet de remplacer le contenu du buffer lors d'une restauration.

La classe **Selection** mémorise deux positions dans le buffer : un indice de début et un indice de fin (il est également possible de les modifier pour sélectionner une partie différente du texte). Le texte situé entre ces bornes est considéré comme sélectionné, et si les deux indices sont égaux alors nous avons à faire à un simple curseur qui peut se déplacer sur le texte.
Dans la v2, sa méthode de restauration ne restaure que la sélection (bornes et ancre).

La classe **Clipboard** contient le texte du presse-papier. Une chaîne de caractères copiée ou coupée est gardée en mémoire dans cette classe. Le presse-papier peut également vérifier s'il est vide ou non.
Cette classe n'évolue pas avec la v2.

La classe **EditorEngine** permet la gestion du buffer de l'éditeur, la sélection de texte ainsi que le presse-papier pour chacune des actions effectuées (copie, coupe, collage, saisie, suppression, déplacement). Cette classe vérifie la validité de la position du curseur ou des bornes de sélection et notifie les observateurs des changements sur l'éditeur.
Dans la v2, cette classe permet de sauvegarder l'état de l'éditeur à un moment donné, puis de restaurer cet état.

L'interface **EditorObserver** propose la signature d'une méthode update pour les différents observateurs utilisés dans notre système.
Cette interface n'évolue pas avec la v2.

L'interface **EditorReader** permet de lire l'état du buffer : son contenu ainsi que la position des bornes de sélection.
Cette interface n'évolue pas avec la v2.

La classe **EditorController** reçoit l'appel à l'action voulue par l'utilisateur correspondant à la commande saisie, et le transmet à EditorEngine.
Dans la v2 sont ajoutées les actions supplémentaires.

Les trois classes suivantes ont été créées pour la v2 uniquement.

L'interface **Command** propose la signature d 'une méthode execute() pour réaliser les différentes actions sur l'éditeur.

La classe **CommandManager** reçoit de EditorController les actions à réaliser et les exécute sur EditorEngine, en précisant pour chacune si elle est annulable. Pour les actions annulables, elle sauvegarde l'état de l'éditeur avant l'exécution et l'ajoute à la pile des actions à défaire si le texte a changé, puis vide la pile des actions à rétablir. Elle gère aussi les commandes de défaire et de rétablir, qui déplacent un état d'une pile à l'autre. Enfin, elle se charge de l'enregistrement et du rejeu : pendant un enregistrement, chaque action exécutée est ajoutée à un script, que la commande de rejeu exécute à nouveau (sans l'enregistrer de nouveau). Elle fournit aussi l'état d'enregistrement et la taille du script (utilisés dans les tests du système).

La classe **EditorMemento** est utilisée dans pour défaire et rétablir. Elle permet de stocker l'état actuel de l'éditeur et de la sélection dans un unique objet pour pouvoir le restaurer plus tard.

Enfin, la classe **Main** crée l'engine, les contrôleurs, la vue et le gestionnaire de clavier. Il connecte la vue à l'engine avec addObserver() et au clavier, puis affiche la fenêtre de l'éditeur pour l'interaction avec l'utilisateur.
Cette classe évolue avec la v2, elle crée également le CommandManager.

I.2.c) Patrons de conception

Patron de conception architectural (MVC) :
Le problème principal était le mélange des données, l'affichage et la saisie de texte nécessitait une interface homme-machine (IHM) pour être testée et exécutée efficacement.

Nous avons donc décidé d'utiliser un système à trois couches MVC (Model, View, Controller). Le modèle contient les données et les règles de fonctionnement de l'éditeur (Buffer, Selection, Clipboard, EditorEngine, EditorReader, EditorObserver et EditorMemento dans la v2), la vue affiche et lit les saisies utilisateur (ConsoleView/SwingView et ConsoleInputHandler/KeyboardHandler) et le contrôleur transmet les actions au modèle (EditorController). Ces trois couches ont été regroupées en packages sur le diagramme de classes et dans l'arborescence du projet (avec le package supplémentaire Command dans la v2).
Grâce à ce patron, nous pouvons tester notre modèle avec des tests JUnit sans avoir besoin d'IHM, car les tests remplacent la couche Vue. La saisie clavier par Swing n'est qu'une possibilité pour les entrées du programme, le contrôleur ne sait pas et n'a pas besoin de savoir d'où vient l'ordre.

Patron de conception comportemental (Observer) :
Le problème principal était le maintien à jour de la vue sachant que le modèle ne doit pas en dépendre pour pouvoir être testé sans vue.
Nous avons donc décidé que la classe EditorEngine possèderait une liste d'EditorObserver et appellerait la méthode update() par le biais de la méthode notifyObservers() après chaque modification de l'éditeur. La classe SwingView/ConsoleView implémente l'interface et lit l'état via EditorReader.
Grâce à ce patron, il est possible d'avoir plusieurs vues sans que le modèle connaisse les vues, car ces vues ne communiquent pas entre elles. Cependant, la méthode update() ne transporte pas de données, donc chaque vue doit relire l'entièreté de l'état de l'éditeur à chaque fois.

Patron de conception comportemental (Memento) :
Le problème principal était de pouvoir revenir à un état antérieur de l'éditeur (défaire) puis le retrouver (rétablir), sans que l'historique connaisse le fonctionnement interne du modèle.
Nous avons donc décidé qu'EditorEngine créerait lui-même des instantanés de son état avec save(), sous la forme d'objets EditorMemento. Un EditorMemento contient le texte ainsi que les trois positions de la sélection (start, end, anchor). EditorEngine rétablit un état avec restore(EditorMemento), qui remet le texte dans le Buffer, la sélection dans Selection, puis notifie les observateurs. Le CommandManager conserve les instantanés dans 2 piles, undoStack et redoStack, sans lire leur contenu. Conformément à notre décision de conception, le presse-papier n'est pas sauvegardé.
Grâce à ce patron, l'historique reste indépendant de la structure du modèle, et le défaire ou le rétablir d'un collage, d'une coupe ou d'une suppression restaure exactement le texte et la sélection d'origine. En contrepartie, chaque instantané contient tout le texte, ce qui consomme plus de mémoire qu'une annulation action par action.

Patron de conception comportemental (Command) :
Le problème principal était de traiter toutes les actions de l'utilisateur de la même manière afin de gérer l'historique et l'enregistrement en un seul endroit, sans modifier le moteur.
Nous avons donc décidé de représenter une action par l'interface Command, qui ne possède qu'une méthode execute(). EditorController crée les commandes à la volée et les transmet à CommandManager.execute(Command, boolean undoable), en précisant si l'action est annulable. Pour la saisie, la suppression, la coupe et le collage, le CommandManager sauvegarde l'état avant l'exécution et ne l'empile sur undoStack que si le texte a changé, puis vide redoStack. La copie et les déplacements sont exécutés sans être empilés. Pendant un enregistrement, chaque commande exécutée est aussi ajoutée au script, et replay() la rejoue en repassant par execute.
Grâce à ce patron, l'historique, l'enregistrement et le rejeu sont centralisés dans CommandManager, le moteur n'a pas à connaître ces fonctionnalités, et une commande peut être mémorisée puis rejouée.

Patron de conception structurel (Façade) :
Le problème principal était que chaque action fait intervenir les classes Buffer, Selection et Clipboard dans un ordre précis tout en respectant l'invariant des positions des bornes de sélection ($0 \leq start \leq end$

≤ length(text)) et en notifiant la vue. Si le contrôleur faisait cela par lui-même, il devrait avoir connaissance de toutes les règles.
Nous avons donc décidé d'exposer une méthode pour chaque action (insert, backspace, moveCursor, extendSelection, cut, copy, paste) par EditorEngine et (undo, redo, record, replay) par CommandManager.
Grâce à ce patron, la vérification de l'invariant et l'envoi de la notification sont garantis en un seul endroit et les tests JUnit passent par un seul point d'entrée.

I.3 - Diagrammes de séquence

Nous avons élaboré un diagramme de séquence pour chaque cas d'utilisation identifié, en nous basant sur les scénarios énoncés précédemment dans la partie I.1.b). Nous avons mutualisé les versions v1 et v2 du système et avons fait figurer les méthodes et classe de la v2 en vert sur les diagrammes.

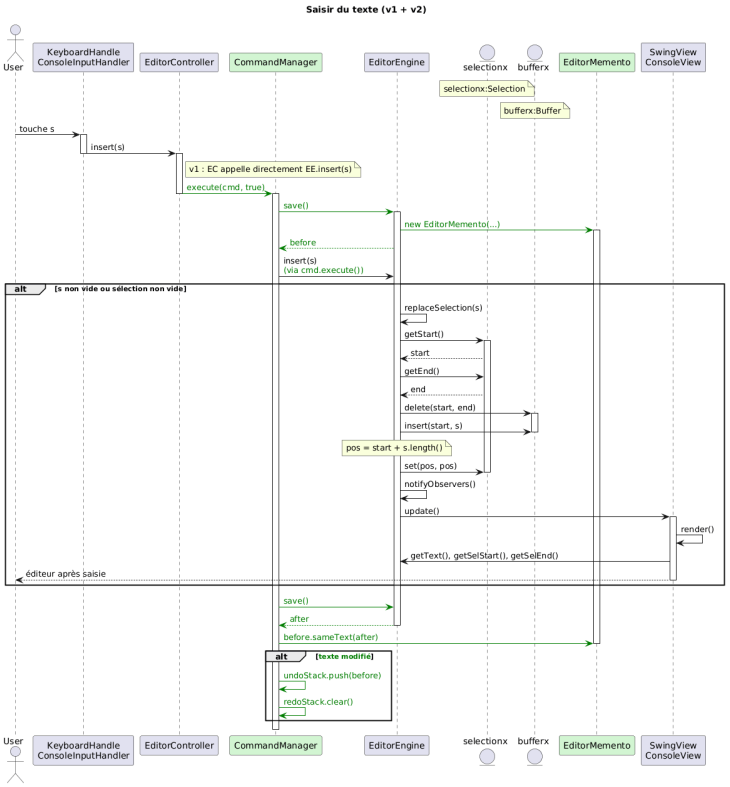


Figure 3 - Diagramme de séquence pour saisir du texte

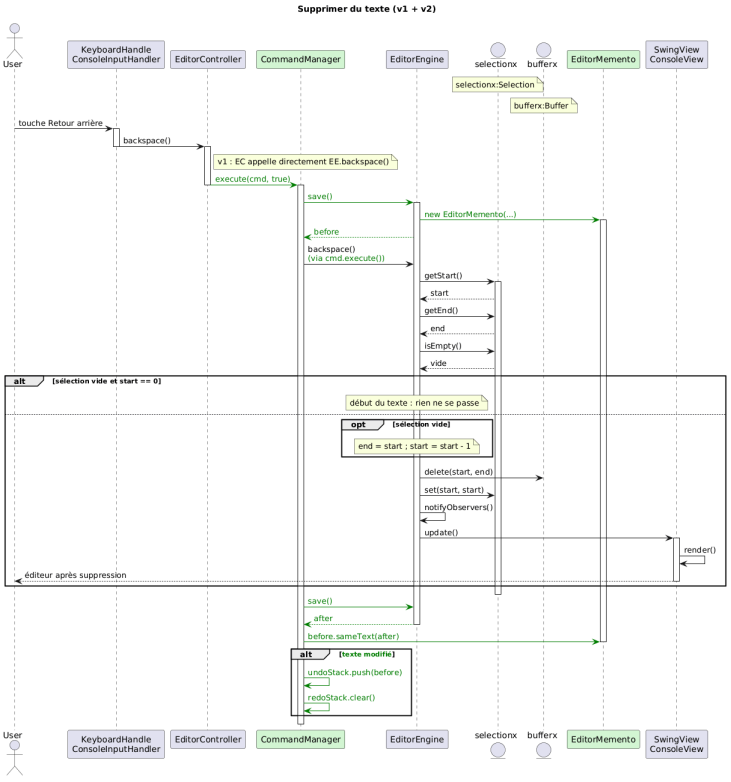


Figure 4 - Diagramme de séquence pour supprimer du texte

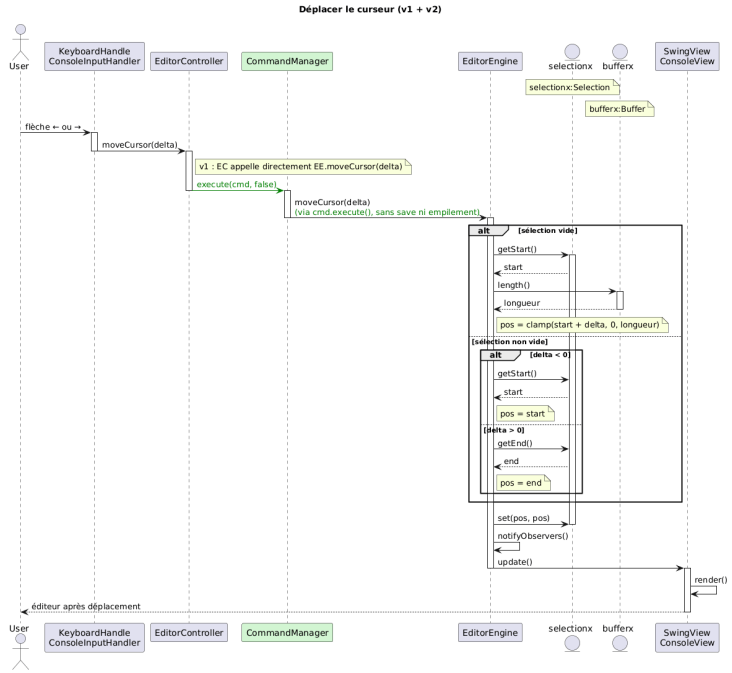


Figure 5 - Diagramme de séquence pour déplacer le curseur

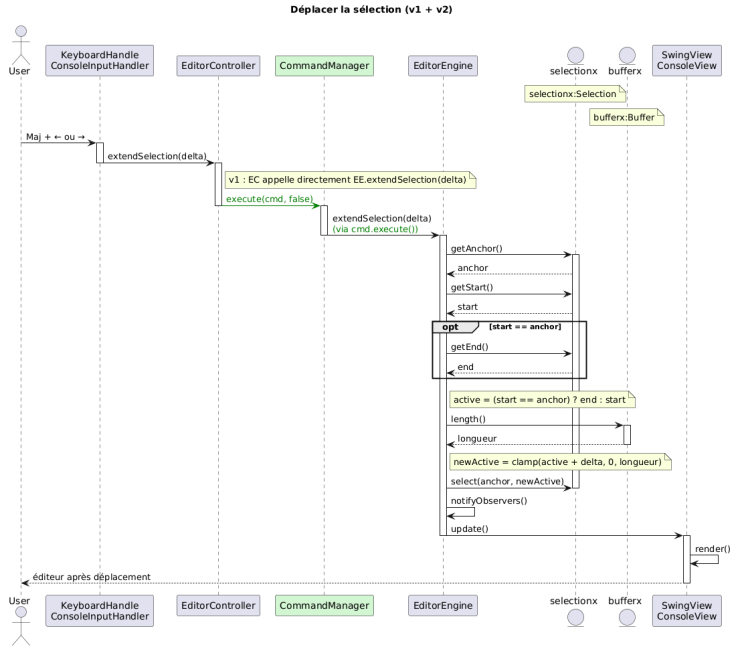


Figure 6 - Diagramme de séquence pour modifier la sélection

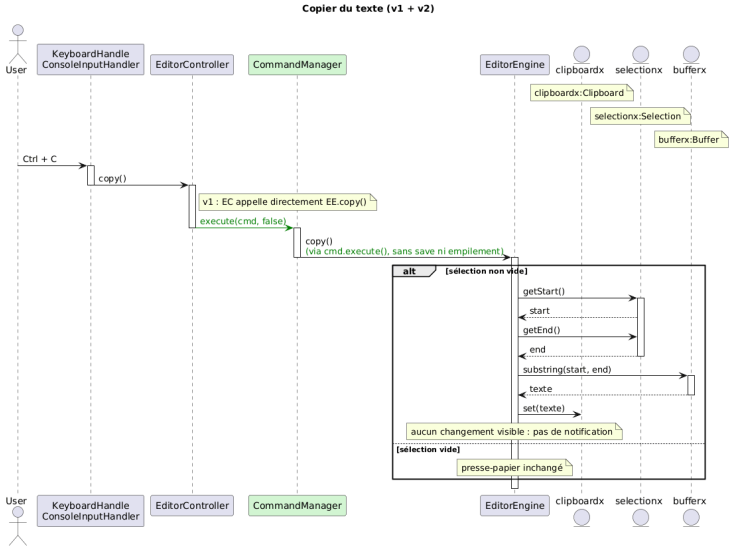


Figure 7 - Diagramme de séquence pour copier du texte

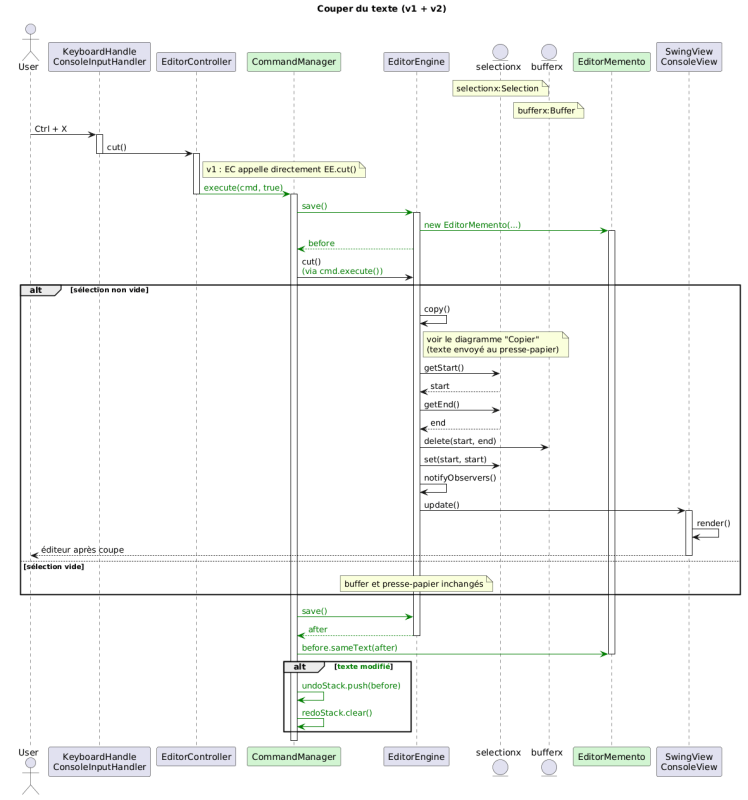


Figure 8 - Diagramme de séquence pour couper du texte

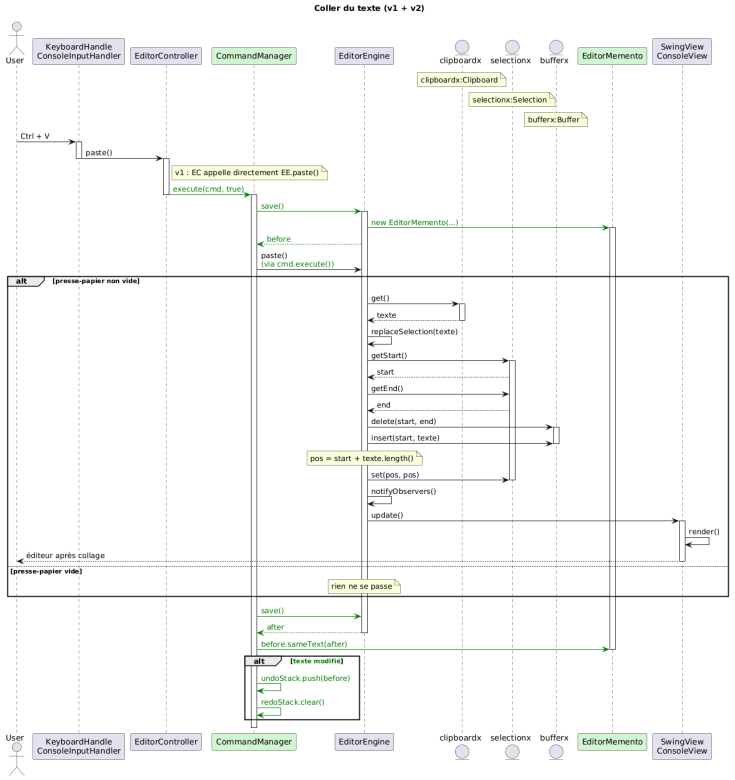


Figure 9 - Diagramme de séquence pour coller du texte

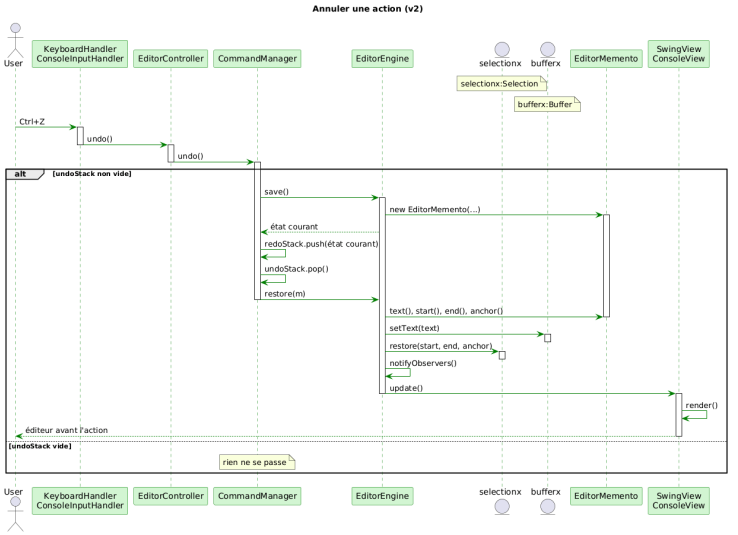


Figure 10 - Diagramme de séquence pour annuler une action

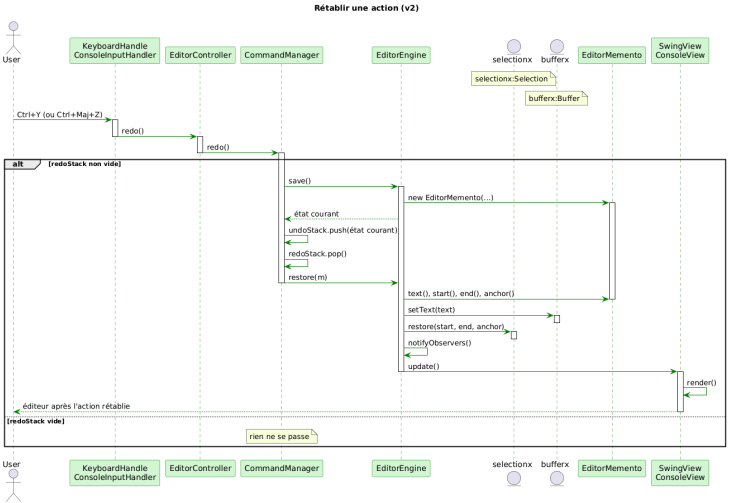


Figure 11 - Diagramme de séquence pour rétablir une action

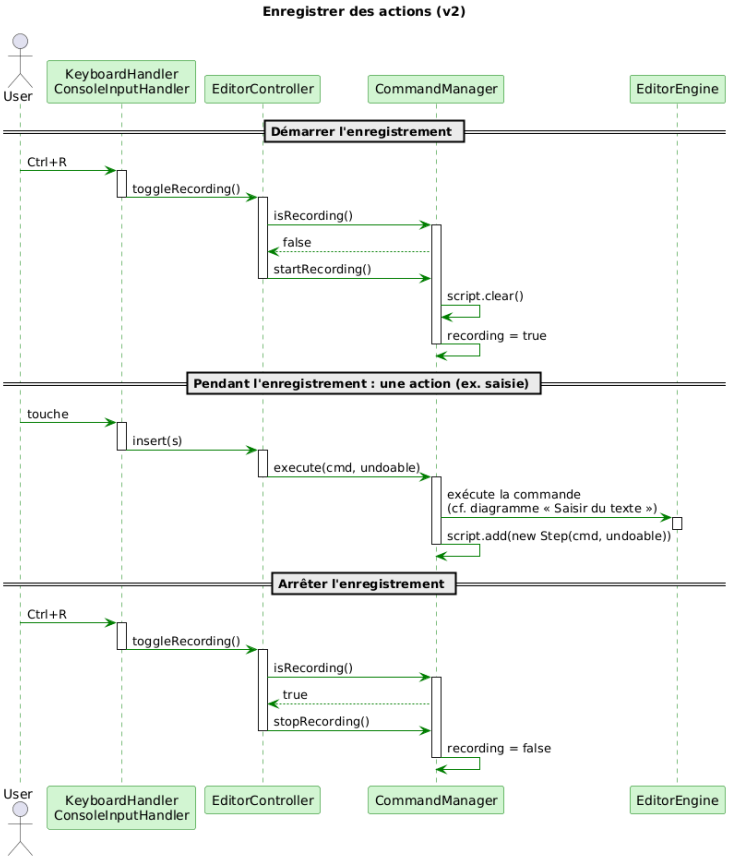


Figure 12 - Diagramme de séquence pour enregistrer des actions

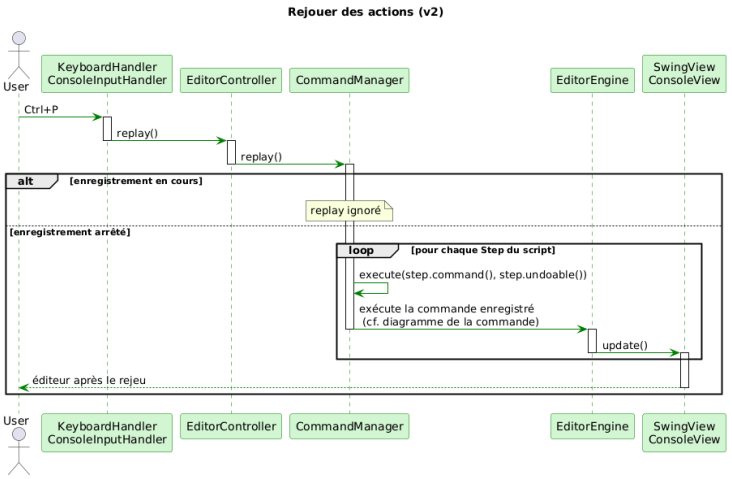


Figure 13 - Diagramme de séquence pour rejouer des actions

I.4 - Diagrammes d'états

Nous avons terminé notre analyse de modélisation et concluons le dossier conceptuel avec l'élaboration des diagrammes d'états. De la même manière, l'apport de la v2 est ajouté en vert sur les diagrammes.

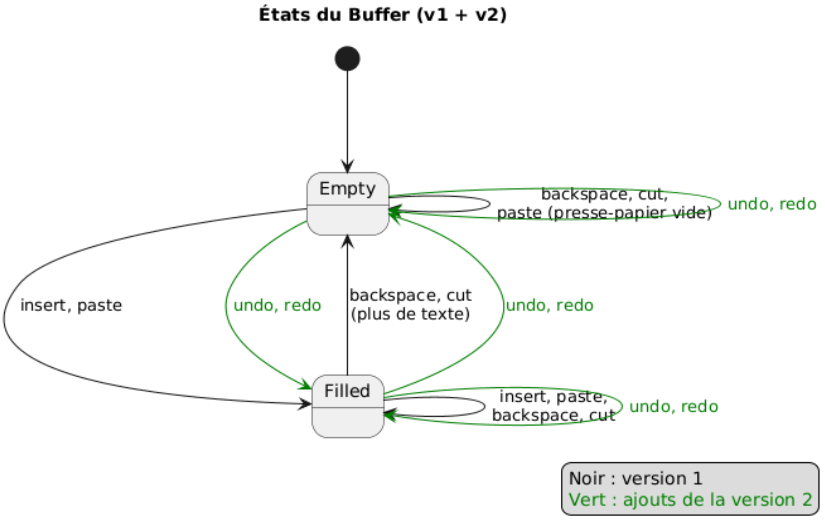


Figure 14 - Diagramme d'états du buffer

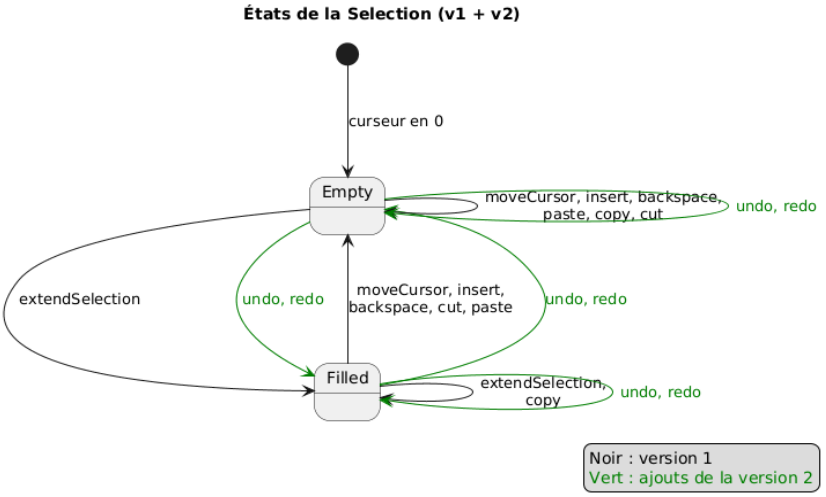


Figure 15 - Diagramme d'états de la sélection

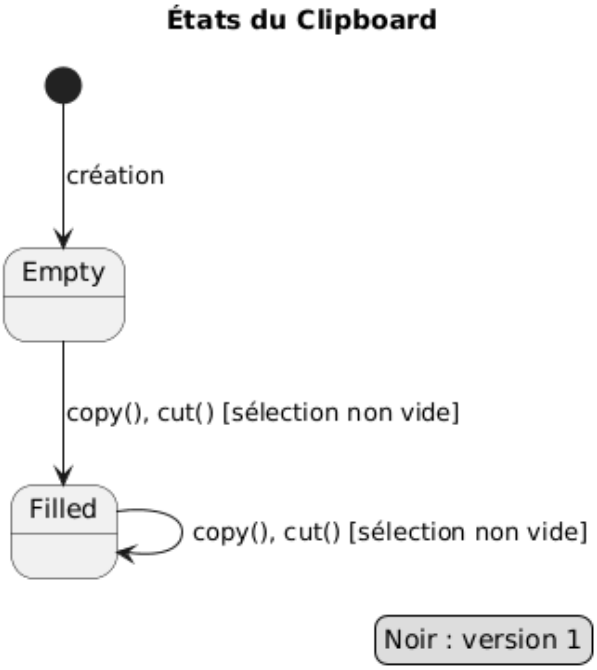


Figure 16 - Diagramme d'états du clipboard

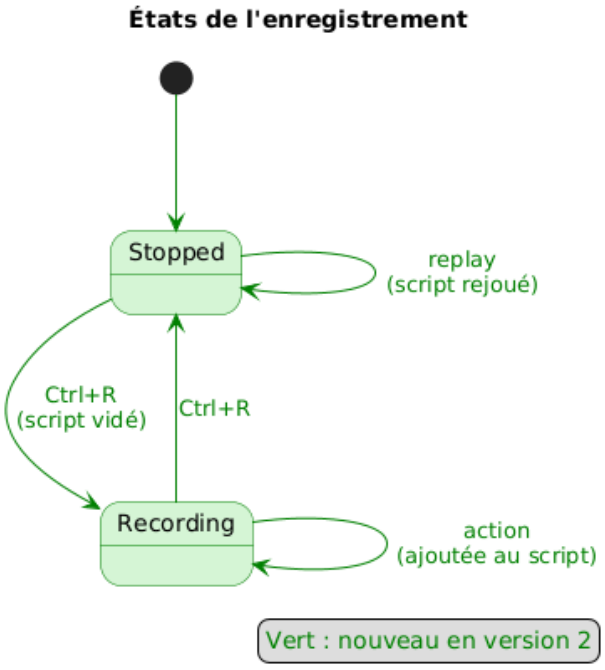


Figure 17 - Diagramme d'états de l'enregistrement

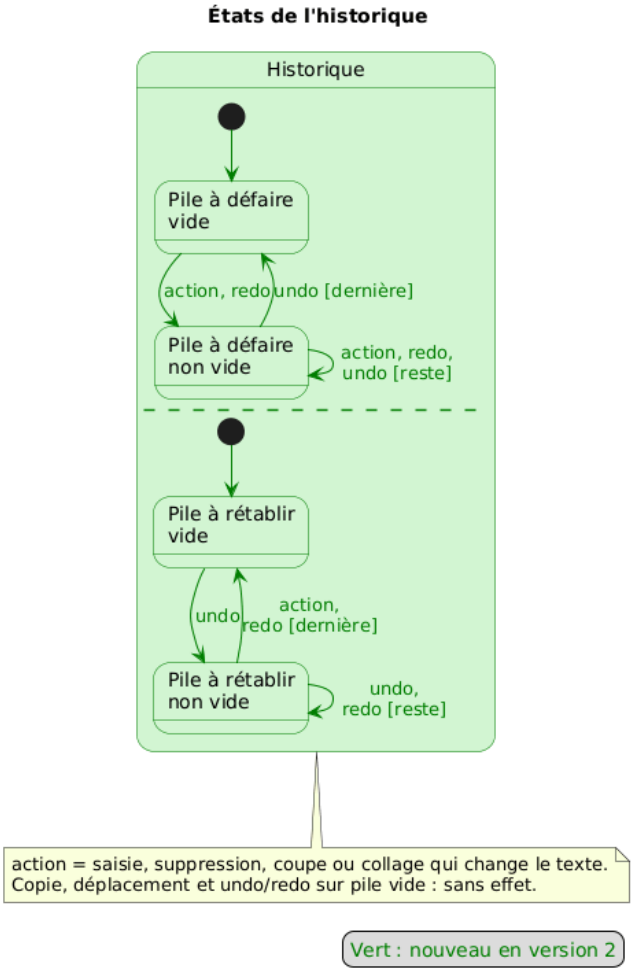


Figure 18 - Diagramme d'états de l'historique

II - Dossier développeur

Ce dossier décrit précisément les deux espaces de travail (workspaces) correspondant aux deux itérations du projet, ainsi que la manière de compiler, lancer et tester chacun d'eux.

II.1 - Environnement et outils

Outils	Usage
Visual Studio Code	Développement Java
JDK	Compilation et exécution
Apache Maven	Gestion du build et des dépendances
JUnit	Tests unitaires
PlantUML	Modélisation (cas d'utilisation, classes, séquences, états)

L'interface homme-machine est graphique et développée avec Swing, la bibliothèque fournie avec le JDK : aucune dépendance externe n'est nécessaire. Ce choix permet de capter directement les raccourcis clavier décrits dans les cas d'utilisation (flèches, Maj, Ctrl+C, Ctrl+X, Ctrl+V, etc.). Le moteur ne dépend d'aucune classe Swing : l'IHM pourrait être remplacée par une interface textuelle sans modifier le moteur.

II.2 - Workspace de la version 1

II.2.a) Organisation du projet

```
v1/
├── pom.xml
├── .gitignore
└── src/
    ├── main/java/editor/
    │   ├── Main.java
    │   ├── controller/
    │   │   └── EditorController.java
    │   ├── model/
    │   │   ├── Buffer.java
    │   │   ├── Clipboard.java
    │   │   ├── EditorEngine.java
    │   │   ├── EditorObserver.java
    │   │   ├── EditorReader.java
    │   │   └── Selection.java
    │   └── view/
    │       ├── KeyboardHandler.java
    │       └── SwingView.java
    └── test/java/editor/model/
        └── EditorEngineTest.java
```

Le code suit une architecture en trois packages (modèle, vue, contrôleur) : model contient l'état de l'éditeur et sa logique, view l'affichage et la capture du clavier, controller fait le lien entre les deux.

II.2.b) Rôle des classes

Package *editor.model* :

Classe	**Rôle**
Buffer	Contient le texte (StringBuilder) : insertion, suppression, lecture d'une portion.
Selection	Mémorise les bornes de la sélection et son ancre. Elle est vide quand start = end.
Clipboard	Contient le texte copié ou coupé.
EditorEngine	Regroupe Buffer, Selection et Clipboard, réalise les actions d'édition et notifie les observateurs.
EditorObserver	Interface Observer (update()).
EditorReader	Interface de lecture seule du moteur, utilisée par la vue.

Package *editor.controller* :

Classe	**Rôle**
EditorController	Transmet les actions de l'utilisateur à EditorEngine.

Package *editor.view* :

Classe	**Rôle**
SwingView	Affiche le texte, la sélection et le curseur, et se redessine à chaque update().
KeyboardHandler	Associe les touches aux méthodes de EditorController.

II.2.c) Compilation, lancement et tests

Depuis le dossier v1/ :

```
mvn compile # compilation
mvn test # tests unitaires
mvn package # génère target/mini-editeur-1.0.jar
java -jar target/mini-editeur-1.0.jar
```

Raccourcis de la version 1 :

Action	**Commande**
Saisir du texte	Touches de caractères (Entrée et Tab inclus)
Supprimer	Retour arrière
Déplacer le curseur	← et →
Étendre la sélection	Maj + ← et Maj + →
Copier / Couper / Coller	Ctrl+C / Ctrl+X / Ctrl+V

II.3 - Workspace de la version 2

II.3.a) Organisation du projet

La version 2 est dans v2/. Elle reprend la structure de la v1 et ajoute un package command.

```
v2/
├── pom.xml
├── .gitignore
└── src/
    ├── main/java/editor/
    │   ├── Main.java                    (modifié)
    │   ├── command/                     (nouveau package)
    │   │   ├── Command.java
    │   │   └── CommandManager.java
    │   ├── controller/
    │   │   └── EditorController.java    (modifié)
    │   ├── model/
    │   │   ├── Buffer.java              (modifié)
    │   │   ├── Clipboard.java
    │   │   ├── EditorEngine.java        (modifié)
    │   │   ├── EditorMemento.java       (nouveau)
    │   │   ├── EditorObserver.java
    │   │   ├── EditorReader.java
    │   │   └── Selection.java           (modifié)
    │   └── view/
    │       ├── KeyboardHandler.java     (modifié)
    │       └── SwingView.java
    └── test/java/editor/command/
        └── CommandManagerTest.java      (nouveau)
```

II.3.b) Evolutions par rapport à la version 1

Classe	**Évolution**
Command (nouveau)	Interface avec execute(). Les commandes sont créées par le contrôleur sous forme de lambdas ou de références de méthodes.
CommandManager (nouveau)	Exécute les commandes et gère les piles défaire/rétablir ainsi que l'enregistrement et le rejeu.
EditorMemento (nouveau)	Instantané de l'état de l'éditeur : texte et sélection, sans le presse-papier.
EditorEngine	Ajout de save() et restore() pour créer et restaurer un instantané.
Buffer, Selection	Ajout d'une méthode de restauration, utilisée par restore().
EditorController	Passe par CommandManager et précise si chaque action est annulable. Ajout de undo, redo, toggleRecording et replay.
KeyboardHandler	Ajout de Ctrl+Z, Ctrl+Y (ou Ctrl+Maj+Z), Ctrl+R et Ctrl+P.
Main	Crée le CommandManager et complète la barre d'aide.

II.3.c) Compilation, lancement et tests

Depuis le dossier v2/ :

```
mvn compile # compilation
mvn test # tests unitaires
mvn package # génère target/mini-editeur-2.0.jar
java -jar target/mini-editeur-2.0.jar
```

Raccourcis ajoutés par rapport à la version 1 :

Action	Commande
Défaire	Ctrl+Z
Rétablir	Ctrl+Y
Démarrer / arrêter l'enregistrement	Ctrl+R
Rejouer	Ctrl+P

II.4) Tests

Nous avons également réalisé des tests unitaires JUnit 5 exécutés par la commande *mvn test*.

Version 1 : EditorEngineTest (12 tests)

Test	**Vérification**
insertPlacesCursorAfterText	Le curseur est placé après le texte saisi
insertReplacesSelection	La saisie remplace la sélection
cursorIsClampedToText	Le curseur ne dépasse pas les extrémités du texte
extendSelectionBothDirectionsAndShrink	La sélection s'étend dans les deux sens, se réduit et fait demi-tour
backspaceDeletesCharBeforeCursor	Suppression du caractère à gauche du curseur
backspaceDeletesSelection	Suppression de toute la sélection
backspaceAtStartOrOnEmptyBufferDoesNothing	Aucun effet en début de texte ou sur un éditeur vide
copyThenPasteReplacesSelectionAndMovesCursorAfter	Copie puis colle, curseur après le texte collé
cutRemovesSelectionAndFillsClipboard	La coupe retire la sélection et va dans le presse-papier
copyAndCutWithEmptySelectionLeaveClipboardUntouched	Copie et coupe avec sélection vide laissent le presse-papier inchangé
pasteWithEmptyClipboardDoesNothing	Coller avec un presse-papier vide ne change rien
observersAreNotifiedOnChange	Les observateurs sont notifiés à chaque modification

Version 2 : CommandManagerTest (10 tests)

Test	**Vérification**
undoRedoCutAndPaste	Défaire/rétablir un collage et une coupe, avec restauration de la sélection
undoGoesBackToTheVeryBeginning	Le défaire remonte jusqu'à l'éditeur vide, le refaire jusqu'à l'état final
newActionClearsRedoStack	Une nouvelle action après un défaire vide la pile de rétablissement
noOpCommandIsNotAddedToHistory	Une action sans effet sur le texte n'est pas empilée
cursorMovesAndCopyAreNotUndoable	Déplacements et copie ne sont pas empilés
undoDoesNotTouchClipboard	Le défaire ne modifie pas le presse-papier
recordAndReplayScript	Enregistrement, rejeu, puis annulation du rejeu
replayIsIgnoredWhileRecording	Le rejeu est ignoré pendant un enregistrement
replayedActionsAreNotRecordedAgain	Les actions rejouées ne sont pas enregistrées de nouveau
engineObserversAreNotifiedOnUndo	Les observateurs sont notifiés lors d'un défaire

Conclusion :

Ce TP nous a permis de concevoir et de développer un mini-éditeur de texte en deux itérations. La version 1 a posé les bases (modèle, vue, contrôleur et actions d'édition) et la version 2 y a ajouté le défaire/rétablir et l'enregistrement/rejeu, sans remettre en cause la version 1 : seul le CommandManager a été ajouté entre le contrôleur et le moteur. Nous avons aussi mis en pratique les patrons Observer (séparation du moteur et de la vue), Memento (sauvegarde de l'état) et Command (exécution centralisée des actions), et construit les diagrammes UML en parallèle du code. Les décisions de conception que nous avons prises en amont nous ont permis de guider l'implémentation et ont pu être vérifiées par des tests unitaires.